

Suivi 3D de pomme de terre — Guide d'utilisation

C'est quoi ?

Ce projet suit la trajectoire d'une pomme de terre qui tombe, en 3D. Deux cameras filment la chute, le code retrouve la position de la pomme de terre dans chaque image, puis reconstruit sa trajectoire dans l'espace.



Installation

1. Cloner le depot

```
git clone <url-du-depot>
cd potato_tracking
```

2. Installer les dependances

```
pip install -r requirements.txt
```

Les dependances sont : `opencv-python` , `numpy` , `matplotlib` , `plotly` , `python-dotenv` .

3. Configurer les dossiers de donnees (optionnel)

Creer ou modifier le fichier `.env` a la racine du projet :

```
DATA_DIR=/home/moi/data/Chute_PdT_Verte
OUTPUT_DIR=/home/moi/data/Chute_PdT_Verte/output
```

Si ce fichier n'existe pas, il faudra passer les chemins en argument (voir plus bas).

4. Verifier la calibration

Le fichier de calibration stereo doit etre present dans `config/camera_parameters.yaml` . Si tu dois le regenerer, lancer :

```
python -m src.calibrate_stereo
```

Structure du code

```
potato_tracking/
  .env                # chemins DATA_DIR et OUTPUT_DIR
  requirements.txt    # dependances Python
  config/
    camera_parameters.yaml  # calibration stereo (K, dist, R, T)
  src/
    config.py         # lecture du .env + parametres du pipeline
    camera.py         # classes CameraIntrinsics et StereoSettings
    segmentation.py   # fonctions de segmentation partagees
    io.py             # lecture/ecriture des images et CSV
    visualize.py      # fonctions de visualisation (2D, 3D, overlay)
    pipeline.py       # pipeline complet (segmentation -> 2D -> 3D -> sorties)
    tracking.py       # point d'entree CLI du pipeline
    segment_object.py # script autonome : segmentation + masques
    trajectoire_2D.py # script autonome : trajectoire 2D par camera
    trajectoire_3D.py # script autonome : trajectoire 3D par triangulation
    calibrate_stereo.py # script autonome : calibration stereo
```

Logique des modules

Module	Contenu
`camera.py`	Chargement/sauvegarde de la calibration. Contient `CameraIntrinsics`
`segmentation.py`	Detection de la pomme de terre dans une image : soustraction de fond
`io.py`	Lecture des dossiers d'images, lecture/ecriture des CSV de trajectoires
`visualize.py`	Generation des graphiques : trajectoire 2D (brute et corrigee), overlay
`pipeline.py`	Orchestre tout le flux : charge la calibration, segmente les deux cam
`tracking.py`	Interface en ligne de commande qui appelle le pipeline. Lit `.env`

Comment lancer ?

Pipeline complet (recommande)

Depuis la racine du projet :

```
python -m src.tracking
```

Cela va :

1. Lire DATA_DIR/CamG/ et DATA_DIR/CamD/
2. Segmenter chaque image (soustraction de fond)
3. Calculer les trajectoires 2D gauche et droite
4. Trianguler pour obtenir la trajectoire 3D
5. Produire les CSV et les graphiques dans OUTPUT_DIR/

Avec des arguments personnalisés



```
python -m src.tracking \
  --data-dir /chemin/vers/donnees \
  --calib config/camera_parameters.yaml \
  --output /chemin/vers/sortie \
  --threshold 25 \
  --min-area 300
```

Argument	Defaut	Description
`--data-dir`	`.env` DATA_DIR ou `data/`	Dossier contenant `CamG/` et `CamD/`
`--calib`	`config/camera_parameters.yaml`	Fichier YAML de calibration
`--output`	`.env` OUTPUT_DIR ou `output/`	Dossier de sortie
`--threshold`	`20`	Seuil de difference pour la soustraction de fond
`--morph-kernel`	`7`	Taille du noyau pour le nettoyage morphologique
`--min-area`	`500`	Aire minimale de l'objet en pixels

Scripts individuels

Chaque etape peut etre lancee separement :

```
# Etape 1 : segmentation (masques + objets extraits)
python -m src.segment_object

# Etape 2 : trajectoires 2D gauche et droite
python -m src.trajectoire_2D

# Etape 3 : trajectoire 3D (necessite les CSV de l'etape 2)
python -m src.trajectoire_3D
```

****Note :**** Les scripts autonomes utilisent leurs propres parametres (en haut de chaque fichier, dans la section `PARAMETRES`).
Penser a les adapter avant de lancer.

Resultats produits

Apres un lancement reussi, le dossier de sortie contient :

```
output/
  trajectoire_gauche/
    trajectoire_2d.csv      # centroides 2D (bruts + corriges)
    trajectoire_2d.png     # graphique 2D
    trajectoire_2d_overlay.png # trace sur l'image de fond
  trajectoire_droite/
    trajectoire_2d.csv
    trajectoire_2d.png
    trajectoire_2d_overlay.png
  trajectoire_3d.csv      # positions 3D (X, Y, Z)
  trajectoire_3d.png      # vues 3D + projections XY/XZ/YZ
  trajectoire_3d.html     # 3D interactive (ouvrir dans un navigateur)
```

Format des CSV

****Trajectoire 2D**** (`trajectoire_2d.csv`) :

frame	cx_brut	cy_brut	cx_corrige	cy_corrige	aire_px
img_001	1024.3	512.7	1024.1	512.5	2340

****Trajectoire 3D**** (`trajectoire_3d.csv`) :

frame	X	Y	Z
img_001	12.5	-45.2	890.3

Les coordonnees 3D sont en millimetres, dans le repere de la camera gauche.

Adapter les parametres de segmentation

Ouvrir le fichier concerne et modifier la section `PARAMETRES` :

**** `segment_object.py` **** — segmentation seule

```
SEUIL_DIFFERENCE = 20      # baisser si l'objet n'est pas detecte
TAILLE_NOYAU_MORPHO = 7    # augmenter pour moins de bruit
AIRE_MINIMALE = 500        # aire minimale en pixels
METHODE = "difference"     # "difference" ou "grabcut"
```

**** `trajectoire_2D.py` **** — trajectoire 2D

```
SEUIL_DIFFERENCE = 20
TAILLE_NOYAU_MORPHO = 7
AIRE_MINIMALE = 500
```

Structure attendue des donnees

Le dossier de donnees (`DATA_DIR`) doit contenir :

```
Chute_PdT_Verte/
  CamG/
    image_0001.tif    # premiere image = fond (scene vide)
    image_0002.tif    # pomme de terre visible
    image_0003.tif
    ...
  CamD/
    image_0001.tif    # premiere image = fond (scene vide)
    image_0002.tif
    image_0003.tif
    ...
```

****Important :**** La premiere image de chaque dossier (ordre alphabetique)

est utilisée comme image de référence (scène sans pomme de terre).
Les images doivent être au format `.tif`.
